Simulation notes

# From Brownian motion to Langevin dynamics

A derivation and a guide to the Python simulation

## Contents

**1 From a coin flip to a Wiener process** **1**

**2 Brownian simulation and mean squared displacement** **2**

**3 The Langevin equation and thermal noise** **3**

**4 Deriving the velocity and position variances** **4**

**5 Mapping the Langevin equations to Python** **5**

**6 Example output from the 10,000-particle run** **6**

**7 How the code builds the theory overlays** **7**

## 1 From a coin flip to a Wiener process

Take $n$ independent fair-coin steps per unit time. Each step $Y_j$ is either $+1/\sqrt{n}$ or $-1/\sqrt{n}$. Its mean is zero and its variance is $\mathbb{E}[Y_j^2] - (\mathbb{E}[Y_j])^2 = 1/n$. Between times $s$ and $t$ there are $M = \lfloor nt \rfloor - \lfloor ns \rfloor$ steps. Independence makes the variances add, so the displacement has mean zero and variance $M/n \to t - s$. The central limit theorem gives a Gaussian limit for this displacement. Sums over disjoint time intervals use different flips, so their limiting increments are independent.

A standard Wiener process $W_t$ starts at $W_0 = 0$, has continuous paths and independent increments, and satisfies

$$W_t - W_s \sim \mathcal{N}(0, t - s), \qquad 0 \le s < t.$$

The coin-flip argument motivates the increment law. The Wiener process also specifies the continuous-time path, which is almost surely nowhere differentiable. Here $\mathcal{N}(\mu, \sigma^2)$ lists the mean and *variance*.

### Why a step uses the square root of time

If $Z \sim \mathcal{N}(0, 1)$, multiplying it by $c$ multiplies its variance by $c^2$. Therefore $\Delta W = \sqrt{\Delta t}\, Z$ has variance $\Delta t$. For example, at $\Delta t = 0.01$ the standard deviation is 0.1. A fresh Gaussian draw per step gives the correct joint distribution at the sampled times. Lines drawn between those times are a visualization of the grid.

## 2 Brownian simulation and mean squared displacement

With $t_j = j\Delta t$, the Brownian loop adds each Gaussian increment to the previous position. Store the initial zero so index $j$ corresponds to time $t_j$.

```
dt = total_time / total_steps
sd = math.sqrt(dt)
times = [step * dt for step in range(total_steps + 1)]
paths = []
for run in range(num_paths):
    position = 0.0
    path = [position]
    for step in range(total_steps):
        z = random.gauss(0, 1)
        position += sd * z
        path.append(position)
    paths.append(path)
```

The mean squared displacement (MSD) uses the same time index from every path. Since $W_0 = 0$ and $\mathbb{E}[W_t] = 0$, the theoretical MSD equals $\mathrm{Var}(W_t) = t$.

$$\widehat{\mathrm{MSD}}(t_j) = \frac{1}{N}\sum_{i=1}^{N}\left[W^{(i)}(t_j) - W^{(i)}(0)\right]^2 = \frac{1}{N}\sum_{i=1}^{N}\texttt{paths[i][j]}^2.$$

Pooling every time index would mix distributions with different widths and include correlated positions from each path. For an endpoint histogram at time $t$, take one value per path and compare with $\mathcal{N}(0, t)$. The 100-path example below uses $\Delta t = 0.01$ and ends at $t = 1$.

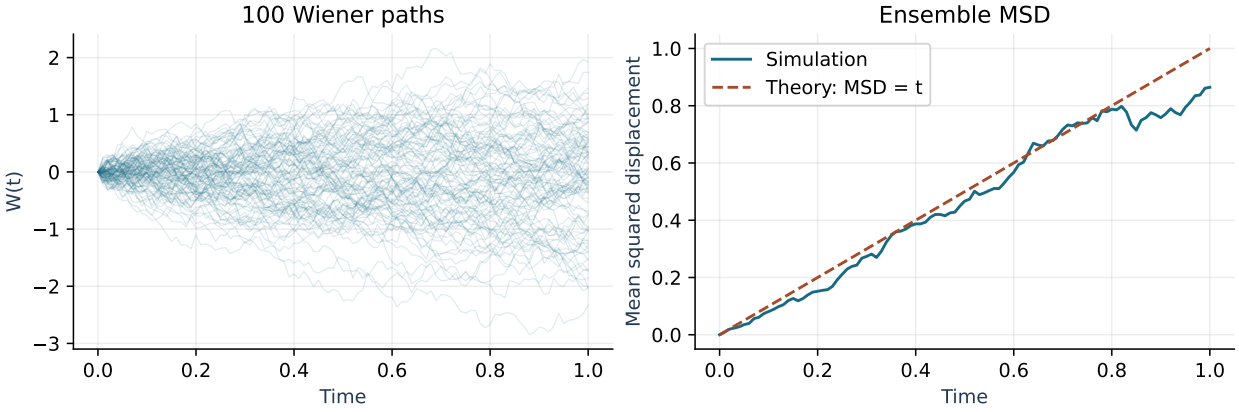



*Figure 1. Brownian paths and ensemble MSD from a seeded run of the 100-path example. At a fixed time the Monte Carlo standard error of the MSD is $t\sqrt{2/N}$, about 0.141 at $t = 1$ for this run. This follows from $\mathbb{E}[W_t^4] = 3t^2$ for a centered Gaussian.*

## 3 The Langevin equation and thermal noise

Consider a free particle of mass $m > 0$ in one dimension, coupled to a heat bath at temperature $T$. The bath produces viscous drag $-\gamma V$, with $\gamma > 0$, and rapidly fluctuating forces. With drag alone, Newton's law gives $m\,\mathrm{d}V/\mathrm{d}t = -\gamma V$, hence $V(t) = V(0)e^{-\gamma t/m}$. Its kinetic energy would decay to zero. At equilibrium, equipartition assigns one quadratic velocity degree of freedom mean energy $k_BT/2$, so

$$\frac{1}{2}m\mathbb{E}[V^2]_{\mathrm{eq}} = \frac{1}{2}k_BT, \qquad \mathrm{Var}(V)_{\mathrm{eq}} = \frac{k_BT}{m} \quad \text{when } \mathbb{E}[V]_{\mathrm{eq}} = 0.$$

The noise represents the net random impulse from the bath. In the white-noise idealization the impulse over a short interval is proportional to the Wiener increment. Write

$$\mathrm{d}X_t = V_t\,\mathrm{d}t, \qquad \mathrm{d}V_t = -aV_t\,\mathrm{d}t + b\,\mathrm{d}W_t, \qquad a = \frac{\gamma}{m}.$$

The velocity is an Ornstein–Uhlenbeck (OU) process. To fix $b$, apply Itô's rule to $V_t^2$. The quadratic variation of the noise contributes $b^2\,\mathrm{d}t$:

$$\mathrm{d}(V_t^2) = (-2aV_t^2 + b^2)\,\mathrm{d}t + 2bV_t\,\mathrm{d}W_t.$$

The stochastic integral has zero mean. At equilibrium the second moment satisfies

$$0 = -2a\mathbb{E}[V^2]_{\mathrm{eq}} + b^2.$$

Equating it to the equipartition value sets the fluctuation–dissipation relation

$$\frac{b^2}{2a} = \frac{k_BT}{m}, \qquad b = \frac{\sqrt{2\gamma k_BT}}{m}.$$

Drag removes kinetic energy and thermal noise replenishes it on average. In the code, `kBT` is the product $k_BT$, and $1/a = m/\gamma$ sets the relaxation time.

### A tool for the variance derivations

For deterministic $f(s)$, the integral $\int_0^t f(s)\,\mathrm{d}W_s$ is the limit of weighted Gaussian increments $\sum_j f(s_j)\Delta W_j$. Each sum is Gaussian with mean zero. Independence eliminates cross terms in its variance, giving $\sum_j f(s_j)^2\Delta s$. Taking the limit yields

$$\mathrm{Var}\left(\int_0^t f(s)\,\mathrm{d}W_s\right) = \int_0^t f(s)^2\,\mathrm{d}s.$$

This deterministic-integrand case of the Itô isometry explains both why the endpoint distributions are Gaussian and why their variances are integrals of squared weights.

# 4 Deriving the velocity and position variances

### Velocity from an integrating factor

Multiply the velocity equation by $e^{at}$. The product rule cancels the drag term, giving $\mathrm{d}(e^{at}V_t) = be^{at}\,\mathrm{d}W_t$. Integration gives

$$V_t = v_0 e^{-at} + b\int_0^t e^{-a(t-s)}\,\mathrm{d}W_s.$$

For fixed $v_0$, $\mathbb{E}[V_t] = v_0 e^{-at}$. Using the squared-weight rule,

$$\mathrm{Var}(V_t) = b^2\int_0^t e^{-2a(t-s)}\,\mathrm{d}s = b^2\int_0^t e^{-2au}\,\mathrm{d}u = \boxed{\frac{b^2}{2a}(1-e^{-2at})}.$$

The factor $2a$ comes from squaring the exponential weight. With $v_0 = 0$, the width grows from zero and approaches the equipartition variance.

### Position from the same Wiener increments

Integrate the velocity SDE itself. Since $\int_0^t V_s\,\mathrm{d}s = X_t - x_0$,

$$V_t - v_0 = -a(X_t - x_0) + bW_t.$$

Substitute the solution for $V_t$ and write $W_t = \int_0^t 1\,\mathrm{d}W_s$. Combining the two integrals before taking a variance keeps their shared noise:

$$X_t = x_0 + \frac{v_0}{a}(1-e^{-at}) + \frac{b}{a}\int_0^t \left(1-e^{-a(t-s)}\right)\mathrm{d}W_s.$$

The first two terms give the mean for fixed $x_0, v_0$. Squaring the kernel and integrating each term gives

$$\begin{aligned}
\mathrm{Var}(X_t) &= \left(\frac{b}{a}\right)^2\int_0^t (1-e^{-au})^2\,\mathrm{d}u \\
&= \left(\frac{b}{a}\right)^2\int_0^t (1-2e^{-au}+e^{-2au})\,\mathrm{d}u \\
&= \boxed{\left(\frac{b}{a}\right)^2\left[t - \frac{2(1-e^{-at})}{a} + \frac{1-e^{-2at}}{2a}\right]}.
\end{aligned}$$

With $x_0 = v_0 = 0$, both endpoint means are zero and the position MSD equals $\mathrm{Var}(X_t)$. For $t \ll 1/a$, $1-e^{-au} \approx au$, so this from-rest MSD grows as $b^2t^3/3$. A thermally equilibrated initial velocity would instead give a leading ballistic term $(k_BT/m)t^2$. For $t \gg 1/a$ the leading term is $2Dt$, with $D = b^2/(2a^2) = k_BT/\gamma$. For the run at $t = 10$, the full expression gives 17.000182, while the leading term is 20.

# 5 Mapping the Langevin equations to Python

Euler–Maruyama takes the drift at the beginning of each interval and uses a Wiener increment with variance $\Delta t$:

$$x_{j+1} = x_j + v_j \Delta t, \qquad v_{j+1} = (1 - a\Delta t)v_j + b\sqrt{\Delta t}\, Z_j.$$

The position line comes first so it uses the old $v_j$. The noise acts on velocity, and position accumulates those velocities. The code evaluates `noise_scale` once, then draws a new $Z_j$ for every particle and step.

```
rng = random.Random(42)
noise_scale = b * math.sqrt(dt)

for particle in range(num_particles):
    x = v = 0.0
    positions = [x]
    for step in range(total_steps):
        z = rng.gauss(0, 1)
        x += v * dt
        v += -a * v * dt + noise_scale * z
        if particle < 100:
            positions.append(x)
    if particle < 100:
        pos_stored.append(positions)
    pos_final.append(x)
    velo_final.append(v)
```

`Random(42)` constructs a pseudorandom generator with a fixed seed for the report run. It is initialized once before the loops, so successive calls use successive draws. The original `random.gauss(0, 1)` uses the same standard normal law through Python's module-level generator. For the current parameters, $a = 1$, $b = \sqrt{2}$, $\Delta t = 10/1000 = 0.01$, and the standard deviation of the noise added to velocity in one step is $b\sqrt{\Delta t} \approx 0.1414$.

`pos_stored` holds 100 complete position paths for plotting. In contrast, `pos_final` and `velo_final` each hold 10,000 values sampled at the same final time. The original `velo_stored` holds entire velocity paths, which the endpoint plots never read.

### Time-step bias and sampling uncertainty

Let $q = 1 - a\Delta t$. Because each new draw is independent of $v_j$, the numerical velocity variance satisfies $S_{j+1} = q^2 S_j + b^2 \Delta t$. For $|q| < 1$ the stationary value is

$$S_\infty = \frac{b^2 \Delta t}{1 - q^2} = \frac{b^2}{2a(1 - a\Delta t/2)}.$$

Here it is about 1.0050, compared with the continuous-time limit 1. More particles reduce sampling fluctuations. A smaller step at fixed final time reduces the discretization bias. For Gaussian endpoints with variance $\sigma^2$, the sample mean has standard error $\sigma/\sqrt{N}$, and the variance estimator has standard error approximately $\sigma^2\sqrt{2/N}$.

## 6 Example output from the 10,000-particle run

The following figures use the pasted Langevin update and parameter values, with the 100-path display, 40-bin histograms, theory overlays, and seed 42. The horizontal histogram shares the position axis with the trajectory plot.

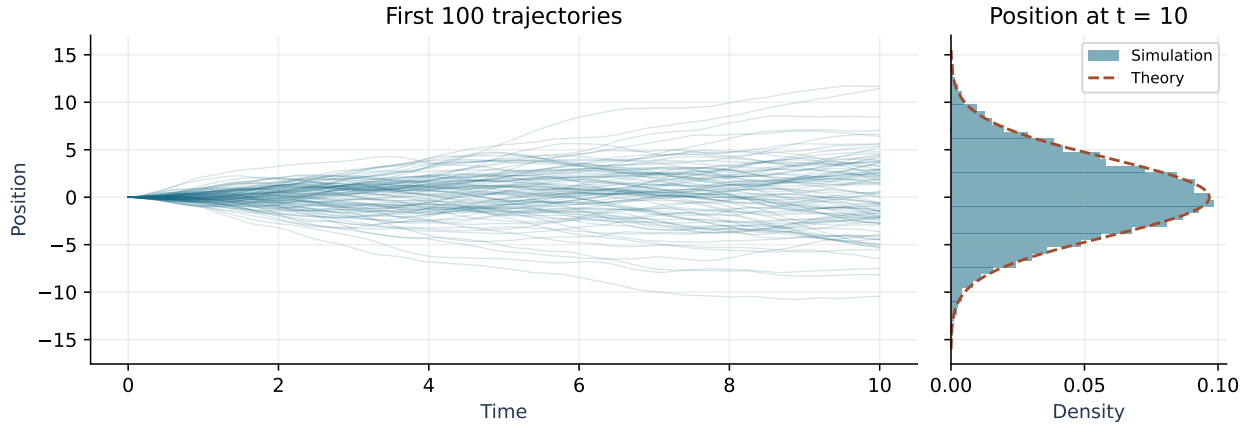



*Figure 2. The first 100 position trajectories and the final-position density from all 10,000 particles. The dashed curve uses the full finite-time position variance.*

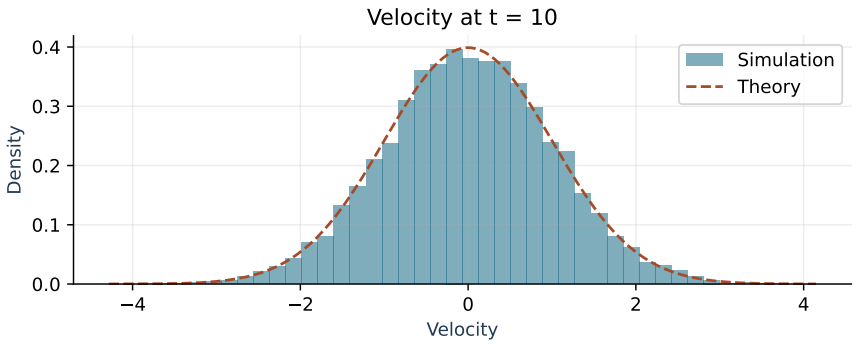



*Figure 3. Final velocities from the same particles, with the continuous-time OU density at $t = 10$.*

| Endpoint | Simulation mean | Theory mean | Simulation variance | Theory variance |
|---|---|---|---|---|
| Position | 0.025002 | 0 | 17.212328 | 17.000182 |
| Velocity | 0.012620 | 0 | 1.006243 | 1.000000 |

*Table 1. Endpoint statistics from the run shown above. The theory column uses continuous time with $x_0 = v_0 = 0$.*

# 7 How the code builds the theory overlays

A centered Gaussian with variance $s^2$ has probability density

$$p(y) = \frac{1}{\sqrt{2\pi s^2}} \exp\left(-\frac{y^2}{2s^2}\right).$$

`NormalDist` expects the standard deviation $s$. Pass the square root of each theoretical variance, then evaluate `pdf` at closely spaced values along the variable's axis.

```
t = times[-1]
var_v = b**2 / (2*a) * (1 - math.exp(-2*a*t))
var_x = (b/a)**2 * (
    t - 2*(1 - math.exp(-a*t))/a
    + (1 - math.exp(-2*a*t))/(2*a)
)
theory_x = statistics.NormalDist(0, math.sqrt(var_x))
theory_v = statistics.NormalDist(0, math.sqrt(var_v))
```

With `density=True`, a bin containing $c$ values and having width $h$ is drawn with density $c/(Nh)$. The total area of the bars is one, matching the scale of the PDF. In the horizontal histogram, the variable is on the vertical axis, so the plotted coordinates are $(p(x), x)$.

```
ax_hist.hist(pos_final, bins=40, density=True,
             orientation="horizontal", alpha=0.5)
low, high = ax_hist.get_ylim()
xs = [low + (high - low) * i / 299 for i in range(300)]
ax_hist.plot([theory_x.pdf(x) for x in xs], xs, "--")
```

For the velocity histogram, `get_xlim()` supplies the velocity range and the curve uses the usual coordinate order $(v, p(v))$. `sharey=True` aligns position values across both panels.

The printed statistics use `statistics.mean` and `statistics.pvariance`. For endpoint values $y_i$ these compute

$$\bar{y} = \frac{1}{N}\sum_i y_i, \qquad \widehat{\mathrm{Var}}(y) = \frac{1}{N}\sum_i (y_i - \bar{y})^2.$$

The MSD about zero satisfies $N^{-1}\sum_i y_i^2 = \widehat{\mathrm{Var}}(y) + \bar{y}^2$. The theoretical means are exactly zero here, while a finite simulation usually has a small nonzero sample mean.

### Reproducing the figures

The source archive includes `make_figures.py` and the generated vector figures. Run `python make_figures.py` with Matplotlib installed to regenerate the plots and print the endpoint statistics. Compile `brownian_langevin.tex` twice with LaTeX to populate the contents.